

LoadLead - Terminal Telematics Integration

Platform Engineering Review

Verdict: STAGED GO | 2026-07-02 | source: docs/telematics-integration-review.md (commit a022210) | prepared by the platform team

Terminal Telematics Integration - Platform Review

Platform Engineering review of the proposed Terminal telematics integration (integrations/terminal module: config, types, client, mappers, migrations, stores, syncService, trackingService, eligibilityService, webhooks, routes; plus the ConnectTelematics React component). Four lenses: backend and API, data and storage, security and compliance, DevOps and infrastructure.

Scope caveat, stated up front: the proposal artifacts (the integrations/terminal module, REPORT.md, and ConnectTelematics.tsx) were not found anywhere on this machine at review time. Searched: this repo, ~/dev/load-lead-frontend (including both Claude worktrees), ~/dev sibling repos, ~/Desktop, ~/Documents, ~/Downloads, and session scratchpads. Per the review brief's fallback, the recommendation checklist below is reconstructed from the brief's own enumeration of the module and its features. Everything about LoadLead's actual platform is grounded in the repo with file paths. Judgments that would require reading the module's code (client conventions, mapper quality, exact interface signatures) are explicitly listed as open questions rather than assumed.

1. Executive summary

- **Feasible now (with one small enabler):** the carrier telematics connection, the equipment roster (Vehicle), and on-demand HOS load eligibility. The platform has the right bones: a proven inbound-webhook pattern with raw-body HMAC verification and idempotency, an integrations adapter convention with fail-closed production guards, and a carrier-resolution primitive that already handles both owner operators and fleet-carrier orgs. The one missing enabler is a KMS envelope-encryption helper for the connection token (small, well-scoped).
- **Blocked or gated:** live tracking with geofenced stop events. It has two hard dependencies the platform does not currently satisfy: (1) there is no background worker or scheduler of any kind - the only recurring job in the codebase is an in-process setInterval that the repo itself labels as something that "should be replaced by an AWS EventBridge rule or a dedicated Lambda" (backend/src/index.ts:247-250); and (2) the append-only stop-events store the tracking service must write to sets event time server-side and rejects caller-supplied timestamps except on corrections (backend/src/services/stopEventService.ts:73-79), so a telematics writer replaying provider timestamps cannot use the store as-is. That second point is a settlements-owned seam and needs their sign-off.

- **Direct conflict with the proposal:** the two new Postgres stores. This platform has no Postgres anywhere - no pg dependency, no RDS in Terraform state, no migration tooling for SQL. Every transactional store is DynamoDB (52 tables under Terraform as of 2026-07-02). Postgres exists only as an unprovisioned analytics-replica plan (docs/database-analytics/analytics-db-spec.md). The stores should be DynamoDB tables following the existing convention, or the integration inherits a database platform, migration tooling, VPC networking, and an ops burden it does not need.
- **Headline recommendation: staged go.** Build the connection, roster, and HOS features now on DynamoDB with a new KMS envelope-encryption helper. Gate tracking behind two explicit enablers (a scheduled worker and a trusted-source extension to the stop-events writer agreed with settlements). Reject the Postgres stores as proposed; note the conflict to the module author.

2. Infrastructure inventory (Phase 2)

2.1 Runtime and deploy

Fact	Value	Source
Node runtime	Node.js 22 on Amazon Linux 2023 (EB solution stack v6.11.1)	infra/terraform/envs/prod/eb-imported.tf:51
EB tier	WebServer only - there is no worker environment	infra/terraform/envs/prod/eb-imported.tf:52
Framework	Express + TypeScript (target ES2020, commonjs)	backend/tsconfig.json, backend/src/index.ts
Deploy path	deploy-backend.sh only: APP_ENV=production assertion, build, stub/_test prune + scan-and-abort, zip, EB update	deploy-backend.sh
Env config	All runtime config via EB environment properties (env vars), read through backend/src/config/environment.ts	config/environment.ts
Prod lockdown	modeResolver + bootGuard fail closed in production; test adapters pruned from the deploy zip	backend/src/services/integrations/(README.md), deploy-backend.sh

Implication: any long-running poller either runs inside the single web tier (stateful, duplicated when EB scales to more than one instance) or needs new infrastructure. There is no second tier to hide work in.

2.2 Background jobs and scheduling

- The ONLY recurring job in the backend is `setInterval(...30_000)` in `backend/src/index.ts:250`, driving `BroadcastService.rebroadcastExpiredLoads()`. The comment directly above it (`index.ts:247-249`) says: "in production this should be replaced by an AWS EventBridge rule or a dedicated Lambda so the EB instance stays stateless."
- No SQS, no EventBridge rules, no cron, no queue consumer anywhere in `backend/src` or `infra/terraform`.
- Lambda precedent exists: the signatures WORM sink is a Lambda with CloudWatch alarms (`infra/terraform/envs/prod/worm-sink.tf`, `observability.tf`), so adding an EventBridge-scheduled

Lambda fits the established infra vocabulary.

Conclusion: a tracking poller has no proper home today. The in-process interval is precedent, but the repo itself flags it as tech debt, and it duplicates on scale-out with no leader election.

2.3 Secrets and encryption

- Secrets are EB environment properties, full stop (deploy-backend.sh prints the expected set: JWT_SECRET, AWS keys, RESEND_API_KEY, etc.). No AWS Secrets Manager, no Parameter Store, no KMS usage in application code (grep across backend/src: zero hits; the only KMS mention in infra is a comment in observability.tf).
- No envelope-encryption or crypto helper exists for data-at-rest fields. DynamoDB tables get default AWS-owned-key encryption at rest, which protects the disk, not the row: anyone with table read access reads the token.
- The connection-token Encryptor seam the proposal expects therefore has nothing to plug into. This is a genuine prerequisite, not paranoia: a Terminal connection token grants standing access to a carrier's entire fleet telemetry, making it the most sensitive per-tenant secret the platform would hold. It is also exactly the artifact a SOC 2 / ISO 27001 auditor asks about ("how are third-party access tokens protected at rest?").

2.4 Webhook ingress

The platform has a strong, proven inbound-webhook pattern - this is the best news in the inventory:

- **Tally webhook** (backend/src/routes/tallyWebhook.ts, mounted in backend/src/index.ts:159-167): mounted with `express.raw({ type: '*/*' })` BEFORE the global `express.json`, so the handler gets the exact bytes; HMAC is verified over those raw bytes (backend/src/services/tallySignature.ts) before any JSON.parse; ingestion is idempotent by responseld. When the secret is absent the endpoint is inert (503), never open (backend/src/config/beta.ts).
- **Didit webhook** (backend/src/services/verification.ts:367+): the global `express.json` is configured with a verify hook that stashes `rawBody` (index.ts:170-171) for HMAC verification. Note one soft spot to NOT copy: the Didit handler bypasses signature verification for events claiming to be test events (verification.ts:370-375) - a header-spoofable trust decision. The Terminal handler should verify first, always.
- Public ingress exists: `api.loadleadapp.com` terminates on the EB environment (ACM cert SAN includes `api.loadleadapp.com`), so a new `/api/webhooks/terminal` route needs no new networking.

2.5 Data layer

- **DynamoDB only.** No pg in backend/package.json. No RDS anywhere in infra/terraform (envs/prod holds DynamoDB, CloudFront, EB, S3, Lambda, SNS/CloudWatch). Table provisioning convention: a block or module entry in infra/terraform (envs/prod/main.tf modules + imported-tables.tf), a matching entry in backend/scripts/createTables.mjs for local/dev, and a key in

backend/src/config/environment.ts. 52 table resources are in prod Terraform state with a clean plan as of 2026-07-02.

- **Postgres exists only on paper:** docs/database-analytics/ analytics-db-spec.md and analytics-db-provisioning.md describe a PostGIS RDS analytics replica (Tier 1 geospatial, Tier 2 telematics "gated on a third-party provider"). It has never been provisioned. There is no SQL migration tooling in the repo. The proposal's migrations/ directory and "two new Postgres stores" have no platform to land on.

- **Append-only stop-events store: exists and is load-bearing.**

backend/src/services/stopEventService.ts writes LoadLead_StopEvents (append-only; corrections are new rows naming correctsEventId; the effective ARRIVAL/DEPARTURE pair is the newest non-superseded event of each type). The schema already carries lat, lng, geofenceMatch, and evidence refs - conceptually a perfect fit for geofenced telematics events. Two contract details matter enormously for the tracking service:

1. `eventAt` is server-set. `buildEvent` throws if a caller supplies `eventAt` without `correctsEventId` (stopEventService.ts:73-79). A telematics writer replaying provider-timestamped events cannot record the true arrival time on a fresh event. Writing corrections instead would be semantically wrong (nothing is being corrected). The store needs a deliberate trusted-source extension (for example, a `source: TELEMATICS` event kind allowed to carry `eventAt`), and that is a settlements-owned change: these events feed the detention/layover money calculation (accessorialCalc.ts, accessorialChargeService.ts). Per docs/TeamOwnership.md RACI, settlements is Accountable for this store; platform and the integration are supplicants here.
 2. Dual-writer semantics. Drivers already write ARRIVAL/DEPARTURE through the UI (routes/accessorials.ts check-in/check-out). A telematics writer for the same stop creates two records of the same physical event. `effectivePair` takes the newest non-superseded event per type, so the calc will not double-bill, but which source wins becomes last-write-wins by `createdAt` - a policy decision (does hardware beat human?) that must be made explicitly, not inherited from sort order.
- Whether the module's StopEventWriter interface matches StopEventService.checkIn/checkOut could not be verified (module absent); given the `eventAt` constraint above, an exact match is unlikely. Open question.

2.6 Auth and identity

- There is no `requireCarrierId` middleware. The actual primitives: `authenticate` + `requireRole(...)` (backend/src/middleware/auth.ts), and carrier resolution via `resolveCarrierIdForUser` (backend/src/routes/factoring.ts): owner-operator profile first, else an ACTIVE OWNER/MANAGER membership in a CARRIER-capability org. Dispatchers and org drivers deliberately do not resolve. The connection feature should reuse this exactly - connecting a fleet's telematics is a management action that binds the org, the same trust shape as factoring. Adapting the module from `requireCarrierId` to this primitive is small but touches the identity seam (identity team consulted, per docs/TeamOwnership.md).

- Drivers are LoadLead_Drivers rows resolved to a carrier of record via backend/src/services/carrierOfRecord.ts. There is no Vehicle table today; the equipment roster is a genuinely new store.

2.7 Frontend build

- Vite + React (frontend-v2). Third-party SDK dependencies are routine (shepherd.js precedent). The Terminal Link SDK as an npm dependency is unremarkable.
- Publishable-key exposure: the established pattern is build-time `VITE_*` env vars (frontend-v2/.env.production carries VITE_API_URL and VITE_GOOGLE_MAPS_API_KEY today). A publishable key fits this exactly. If runtime (no-rebuild) configuration is wanted instead, the precedent is a server-driven flag endpoint like GET /api/beta/status - either works; the build-time var is one line.
- The ConnectTelematics component itself could not be reviewed (absent).

2.8 Observability

- Logging is console-based (backend/src/utils/logger.ts) and lands in CloudWatch via EB log streaming. No error tracker (no Sentry), no APM.
- Alarm plumbing exists: SNS topic `loadlead-prod-ops-alerts` + CloudWatch alarms for the WORM-sink Lambda (infra/terraform/envs/prod/observability.tf). One operational hole the file itself documents: the SNS topic has NO subscribers (observability.tf:24-26). Until someone runs the documented `aws sns subscribe` command, every alarm fires into the void. That is a five-minute prerequisite for any "alert on webhook failure" story.

2.9 Existing telematics seam (do not duplicate)

- backend/src/services/telematics.ts is an honest MVP gate: `TELEMATICS_PROVIDER` env var means "connected", empty means "not connected", and the admin fleet feed then labels driver-app heartbeats as NOT telematics (backend/src/middleware/driverLocation.ts enforces a 15-minute freshness window on the heartbeat). Its own comment says the integrations adapter pattern "would normally own this" once a real provider exists. The Terminal module is that real provider: it should replace this env-var gate (per-carrier connection state supersedes a global flag), and the admin integrations panel (frontend-v2/src/pages/admin/AdminSettings.tsx) plus `api.adminFleetFeed` already surface connection status to flip over.

3. Recommendation checklist (Phase 1) and per-item assessment (Phase 3)

Checklist reconstructed from the review brief (REPORT.md absent; see scope caveat). Feasibility ratings: **NOW** (implement now), **INFRA** (needs new infrastructure first), **BLOCKED** (needs decision/verification first).

#	Item	Feasibility	Effort	What exists	Key gaps / risks
1	Carrier telematics connection (Link flow, token storage, connect/disconnect routes)	NOW after item 10	M (3-5 d)	Webhook+adapter conventions, resolveCarrierIdForUser, express routing, integrations fail-closed pattern	Encryptor has nothing to plug into until the KMS helper exists; requireCarrierId does not exist (adapt to resolveCarrierIdForUser); store must be DynamoDB not Postgres
2	ConnectTelematics React component + publishable key	NOW	S (1-2 d)	Vite SDK deps routine; VITE_* pattern (.env.production)	Component unreviewable (absent); key is build-time baked - fine for a publishable key
3	Equipment roster - Vehicle sync	NOW	S-M (2-4 d)	DynamoDB table convention (terraform + createTables.mjs + environment.ts); Tally webhook idempotency pattern for sync events	New LoadLead_Vehicles table; upsert idempotency by provider vehicle id; no Vehicle concept exists anywhere today so no collision
4	Live tracking + geofenced stop events (Latest Vehicle Location -> StopEventWriter)	INFRA + BLOCKED	L (8-12 d after enablers)	Stop-events store exists with lat/lng/geofenceMatch fields; detention calc consumes it; admin fleet map exists to display	No worker/scheduler (2.2); eventAt server-set conflict (2.5.1) needs settlements sign-off; dual-writer precedence policy (2.5.2); rate/cost budget unknown
5	Load eligibility - HOS Available Time	NOW after item 1	M (3-5 d)	Offer/acceptance flow to gate; matching rule consolidation point; on-demand pull needs no worker	Fail-open vs fail-closed policy when HOS unavailable (safety vs availability - product call); cache TTL; provider rate limits unknown
6	Two new Postgres stores + migrations/	BLOCKED (as proposed)	n/a	Nothing - no Postgres, no RDS, no SQL migration tooling	Direct conflict with platform convention; re-target to DynamoDB (S per store); PostGIS analytics replica is a separate, unprovisioned roadmap item (docs/database-analytics/)
7	Webhook handler + routes	NOW (skeleton) / BLOCKED (verification)	S (1-2 d)	Full pattern proven: raw-body mount before express.json, HMAC over exact bytes, idempotent ingest, inert-when-unconfigured (tallyWebhook.ts, tallySignature.ts, index.ts:159-171)	CONFIRM: Terminal's signature scheme (header name, algorithm, timestamp tolerance) must come from vendor docs - blocks webhook ingest only, not polling; do NOT copy the Didit test-event bypass
8	Connection-token encryption (Encryptor)	INFRA (prerequisite)	S (1-2 d)	Nothing in app code; DynamoDB default at-rest	New utils/crypto envelope helper (KMS GenerateDataKey)

#	Item	Feasibility	Effort	What exists	Key gaps / risks
	seam)			encryption insufficient for this class of secret	+ AES-256-GCM), KMS key in terraform, EB instance-role kms:Decrypt grant
9	Real-time polling for tracking	INFRA	M (4-6 d)	setInterval precedent exists but repo flags it as debt (index.ts:247-250); Lambda + alarm precedent (worm-sink.tf)	Choose: EventBridge Schedule -> Lambda (fits existing infra vocabulary, stays out of the web tier) vs EB worker tier (new tier class) vs in-process (rejected: stateful, duplicates on scale-out)
10	Rate-limit and cost budget for location + HOS calls	BLOCKED (data needed)	S (analysis)	Nothing - no vendor pricing in repo	CONFIRM against vendor docs: per-call pricing, rate limits, burst; design lever: poll per-carrier batch endpoints, cache latest-location in DynamoDB, HOS on-demand only
11	Observability for connection health + webhook failures	NOW	S (1-2 d)	SNS ops-alerts topic + CloudWatch alarm pattern (observability.tf); console logging via EB	Topic has zero subscribers (observability.tf:24-26) - subscribe first or all alerting is theater; add alarms: webhook 5xx, sync failures, connection-disconnected events
12	CONFIRM: webhook signature scheme	BLOCKED (vendor docs)	hours	n/a	Blocks item 7 ingest; polling-based sync is the fallback until confirmed
13	CONFIRM: Driver, Trailer, Trip, Connection, Sync field sets	BLOCKED (vendor docs)	hours-days	DynamoDB is schemaless - store schemas tolerate field-set drift far better than the proposed SQL migrations would	Blocks mapper finalization; does NOT block starting items 1-3 since DynamoDB rows can carry the raw payload plus mapped fields

Notes on the CONFIRM items: none of them block the connection feature architecturally. The signature scheme gates only webhook-push ingestion (poll-based sync works meanwhile). The field sets gate mapper completeness, and the platform's DynamoDB convention actually de-risks this compared to the proposal's SQL migrations: store the vendor payload verbatim alongside the mapped projection and field-set corrections become code changes, not schema migrations.

4. Cross-cutting infrastructure gaps (Phase 4)

Gap	Exists today?	Missing	Work to close
Envelope encryption / KMS for connection tokens	No (2.3)	KMS key, crypto helper, IAM grant	S: aws_kms_key + alias in envs/prod, backend utils/crypto.ts (GenerateDataKey + AES-256-GCM, key id in ciphertext envelope), kms:Decrypt/GenerateDataKey on the EB instance role, unit tests
Webhook ingress with signature + idempotency	Yes, as a pattern (2.4)	Terminal-specific endpoint, secret env, event-id idempotency store	S: clone the Tally mount pattern (raw body BEFORE express.json), verify-always (no test bypass), conditional-put dedupe on provider event id
Scheduled polling worker	No (2.2)	Any scheduler at all	M: EventBridge Schedule -> Lambda invoking an internal batch endpoint or running the poll directly; terraform module mirroring worm-sink.tf; alarm on errors. This also retires the flagged rebroadcast setInterval debt for free
Rate-limit and cost budget	No	Vendor pricing + limits	S analysis once docs are in hand; design already biases cheap: on-demand HOS, batched location polls, DynamoDB latest-location cache
Connection-health observability	Partial (2.8)	SNS subscriber, integration alarms, connection-state surfacing	S: subscribe ops email to loadlead-prod-ops-alerts (documented one-liner in observability.tf), CloudWatch alarms for webhook failures and sync lag, extend the AdminSettings integrations panel which already shows telematics status

5. Prioritized roadmap (Phase 5)

Scoring lenses: product impact (visibility, detention accuracy, safety), effort and risk, dependency order, infrastructure readiness.

NOW (unblocked or one small enabler away)

Order	Item	Size	Rationale
1	SNS subscriber + KMS envelope helper (items 11 pre-step, 8)	S	Two tiny enablers everything else leans on; zero product risk
2	Carrier connection + ConnectTelematics (items 1, 2)	M	The gateway feature; everything else is dead until a carrier can connect; DynamoDB store, resolveCarrierIdForUser gating
3	Equipment roster (item 3)	S-M	Cheap, immediately visible in the carrier dashboard, exercises the sync path end to end
4	HOS eligibility, on-demand (item 5)	M	Highest safety-per-dollar; needs no worker; product must decide fail-open vs fail-closed when HOS is unavailable
5	Webhook endpoint skeleton (item 7)	S	Build inert-until-configured like Tally; flip on once the signature scheme is CONFIRMED

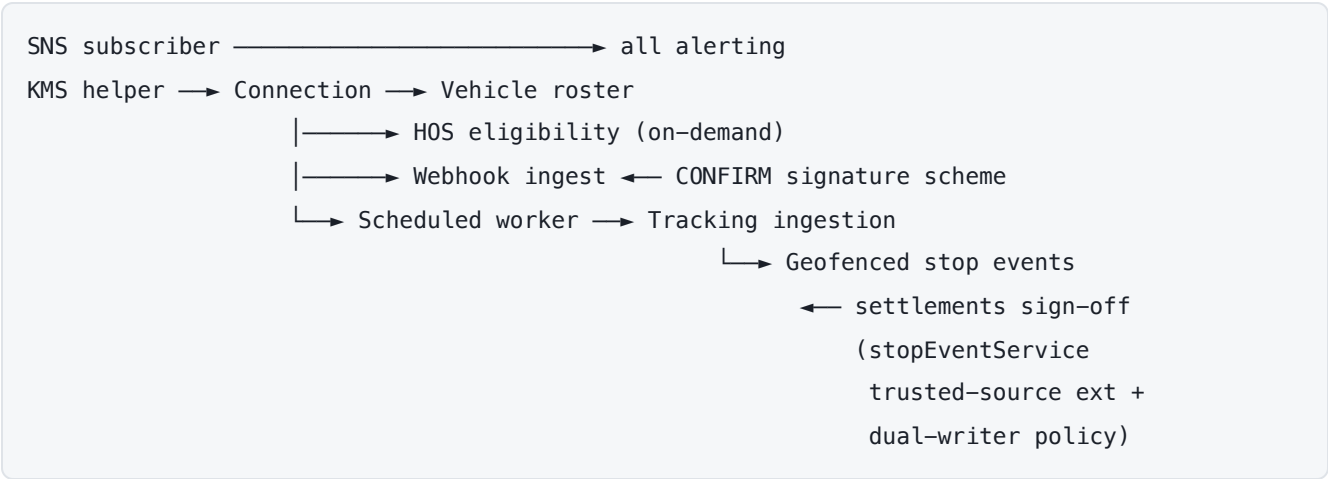
NEXT (after the enablers land)

Order	Item	Size	Rationale
6	Scheduled worker (item 9)	M	EventBridge -> Lambda; also retires the rebroadcast setInterval debt the repo already flags
7	Tracking ingestion + latest-location cache (item 4a)	M	Live map data with clear "telematics" labeling, replacing the heartbeat-only feed (services/telematics.ts)
8	Geofenced stop events -> detention (item 4b)	M-L	The money feature: hardware-attested arrival/departure feeding accessorialCalc; REQUIRES the settlements-approved trusted-source extension to stopEventService and an explicit dual-writer precedence policy

LATER

Item	Size	Rationale
PostGIS analytics replica (per docs/database-analytics/analytics-db-spec.md Tier 1)	L	The legitimate home for Postgres in this architecture; unrelated to transactional stores
Trips / Trailers / fuel / CSA expansion (analytics spec Tier 2)	L	Gated on connected-carrier volume proving the integration earns its keep

Dependency graph



6. Risks and open questions

1. **The module itself is unreviewed.** REPORT.md, the integrations/terminal code, and ConnectTelematics were not found on this machine (paths searched listed in the scope caveat). Every module-internal judgment - client retry/error conventions, mapper correctness, the exact StopEventWriter signature, route shapes - is an open question until the files land in the repo. This review grounds the platform side; a code review of the module must follow.

2. **CONFIRM (vendor docs): webhook signature scheme.** Header, algorithm, timestamp tolerance, replay window. Blocks webhook ingest only; polling covers sync meanwhile.
 3. **CONFIRM (vendor docs): Driver, Trailer, Trip, Connection, Sync field sets.** Blocks mapper freeze, not feature start (DynamoDB tolerates drift; store raw payload + projection).
 4. **Vendor pricing and rate limits** for Latest Vehicle Location and HOS Available Time. Unknown; caps the polling cadence and therefore geofence latency. Needed before sizing the worker's schedule.
 5. **Stop-events trusted-source extension** is a settlements decision, not a platform one. These rows move money (detention). The extension (allow eventAt from a TELEMATICS source, plus the precedence policy against driver check-ins) must be designed and tested by settlements with platform consulted, per docs/TeamOwnership.md.
 6. **Dual-writer precedence** (telematics vs driver check-in for the same stop) is a product/settlements policy call. The current newest-wins sort is an accident of implementation, not a decision.
 7. **HOS fail-open vs fail-closed** when the provider is down or the carrier disconnects mid-offer: safety says closed, liquidity says open. Product call; must be explicit in eligibilityService.
 8. **PII and consent.** Vehicle location and HOS are driver-behavioral data. The platform has consent plumbing precedent (ESIGN acceptance rows, accessorial acknowledgments - accessorialPolicyService.ts) but nothing covering telematics data collection from drivers of a connected carrier. For SOC 2 / ISO 27001 scope: data inventory entry, retention rule (the latest-location cache should be short-retention, not append-only forever), and a consent/notice artifact for drivers. Open design item.
 9. **Token lifecycle.** Disconnect/reconnect flows must revoke and re-issue the stored token; alarms on sync-auth failures (401s from the vendor) are the disconnect detector. Covered by item 11 but called out because silent token death is the classic telematics-integration failure mode.
 10. **Didit test-event bypass as an anti-pattern** (verification.ts:370-375): header-spoofable signature bypass. The Terminal webhook must not copy it. Flagged also as a pre-existing hardening item for identity.
-

7. Recommendation

Staged go.

- **Go now** on the connection, roster, and HOS features, re-targeted to DynamoDB stores and the platform's existing conventions (Tally-style webhook mounting, resolveCarrierIdForUser gating, integrations fail-closed pattern), preceded by the two small enablers: the KMS envelope helper and an SNS subscriber. This is roughly two to three weeks of well-understood work with no architectural risk.
- **Gate tracking** behind the scheduled worker (EventBridge -> Lambda, which also pays down debt the repo already flags) and the settlements-approved trusted-source extension to the stop-events

store. Do not let the integration write money-bearing events through a side door or a corrections hack.

- **Hold the Postgres stores** as proposed. They conflict with the actual platform. Postgres enters this architecture through the existing analytics-replica spec or not at all.
- Do not merge the module until it exists in the repo and has had a code review against the conventions cited here; this document is the platform half of that review.